

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа №3

Выполнила:

Маркозубова Анастасия

К3339

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

1 Задача

- реализовать Dockerfile для каждого сервиса;
- написать общий docker-compose.yml;
- настроить сетевое взаимодействие между сервисами.

2 Ход работы

Для каждого микросервиса был добавлен отдельный Dockerfile. Все Dockerfile построены по единому принципу: сначала выполняется установка зависимостей и сборка TypeScript-кода, затем в финальный образ копируются только собранные файлы dist и production-зависимости.

- Базовый образ: node:22-bookworm.
- Рабочая директория внутри контейнера: /app.
- Зависимости устанавливаются командой npm ci на основе package-lock.json.
- Сборка выполняется командой npx tsc -p tsconfig.json.
- Запуск сервиса выполняется командой node dist/app.js.

Пример структуры Dockerfile для сервиса:

```
FROM node:22-bookworm AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY auth-service/tsconfig.json ./
COPY auth-service/src ./src
RUN npx tsc -p tsconfig.json
RUN npm prune --omit=dev
```

```
FROM node:22-bookworm
WORKDIR /app
COPY --from=build /app/node_modules ./node_modules
COPY --from=build /app/dist ./dist
EXPOSE 8001
CMD ["node", "dist/app.js"]
```

Для запуска всех компонентов приложения был написан общий docker-compose.yml. В нем описаны контейнеры шести backend-сервисов, шести экземпляров PostgreSQL и RabbitMQ с web-интерфейсом управления.

- RabbitMQ опубликован на портах 5672 и 15672.
- API-сервисы опубликованы на портах 8001-8006.
- PostgreSQL-базы опубликованы на портах 15431-15436.
- Для баз данных используются именованные volumes, чтобы данные сохранялись между перезапусками.
- Для RabbitMQ и PostgreSQL настроены healthcheck-проверки.

Контейнер	Внешний порт	Внутренний порт	Назначение
rabbitmq	5672	5672	AMQP-подключение сервисов
rabbitmq	15672	15672	RabbitMQ Management UI
auth-service	8001	8001	Auth API
profile-service	8002	8002	Profile API
resume-service	8003	8003	Resume API
vacancy-service	8004	8004	Vacancy API
application-service	8005	8005	Application API
reference-service	8006	8006	Reference API

Все контейнеры подключены к общей bridge-сети job-board-network. Благодаря этому сервисы обращаются друг к другу не через localhost, а по DNS-именам контейнеров Docker Compose.

- AUTH_SERVICE_URL=http://auth-service:8001/api
- PROFILE_SERVICE_URL=http://profile-service:8002/api
- RESUME_SERVICE_URL=http://resume-service:8003/api
- VACANCY_SERVICE_URL=http://vacancy-service:8004/api
- APPLICATION_SERVICE_URL=http://application-service:8005/api

- REFERENCE_SERVICE_URL=http://reference-service:8006/api
- RABBITMQ_URL=amqp://rabbitmq:5672

Для приложений APP_HOST установлен в значение 0.0.0.0, чтобы Node.js-серверы принимали подключения извне контейнера. При этом Swagger отображает localhost для удобного тестирования API из браузера.

RabbitMQ используется для обмена доменными событиями между микросервисами. Каждый сервис подключается к exchange job_board.events и создает собственную очередь вида <service>.events.

Очередь	Сервис-владелец
auth.events	auth-service
profile.events	profile-service
resume.events	resume-service
vacancy.events	vacancy-service
application.events	application-service
reference.events	reference-service

При проверке после обновления данных компании в логах сервисов появились сообщения о получении события profile.company.updated от profile-service. Это подтверждает, что событие было опубликовано в RabbitMQ и доставлено другим сервисам. В web-интерфейсе RabbitMQ счетчики сообщений остаются равны нулю, потому что сервисы сразу получают сообщения и подтверждают их обработку через ack.

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
lab3-application-db-1	postgres:16	"docker-entrypoint.s..."	application-db	7 hours ago	Up 1 second (health: starting)	5432/tcp
lab3-application-service-1	lab3-application-service	"docker-entrypoint.s..."	application-service	6 hours ago	Up 1 second	8005/tcp
lab3-auth-db-1	postgres:16	"docker-entrypoint.s..."	auth-db	7 hours ago	Up 1 second (health: starting)	5432/tcp
lab3-auth-service-1	lab3-auth-service	"docker-entrypoint.s..."	auth-service	6 hours ago	Up 1 second	8001/tcp
lab3-profile-db-1	postgres:16	"docker-entrypoint.s..."	profile-db	7 hours ago	Up 1 second (health: starting)	5432/tcp
lab3-profile-service-1	lab3-profile-service	"docker-entrypoint.s..."	profile-service	6 hours ago	Up 1 second	8002/tcp
lab3-rabbitmq-1	rabbitmq:3-management	"docker-entrypoint.s..."	rabbitmq	7 hours ago	Up 1 second (health: starting)	5672/tcp, 15672/tcp
lab3-reference-db-1	postgres:16	"docker-entrypoint.s..."	reference-db	7 hours ago	Up 1 second (health: starting)	5432/tcp
lab3-reference-service-1	lab3-reference-service	"docker-entrypoint.s..."	reference-service	6 hours ago	Up 1 second	8006/tcp
lab3-resume-db-1	postgres:16	"docker-entrypoint.s..."	resume-db	7 hours ago	Up 1 second (health: starting)	5432/tcp
lab3-resume-service-1	lab3-resume-service	"docker-entrypoint.s..."	resume-service	6 hours ago	Up 1 second	8003/tcp
lab3-vacancy-db-1	postgres:16	"docker-entrypoint.s..."	vacancy-db	7 hours ago	Up 1 second (health: starting)	5432/tcp
lab3-vacancy-service-1	lab3-vacancy-service	"docker-entrypoint.s..."	vacancy-service	6 hours ago	Up 1 second	8004/tcp

3 Вывод

В ходе работы были закреплены навыки контейнеризации backend-приложений, описания многоконтейнерной инфраструктуры через Docker Compose и настройки сервисного взаимодействия внутри Docker-сети. Использование Docker Compose позволило запускать все компоненты приложения одной командой и обеспечить одинаковую среду выполнения для всех микросервисов.